Decentralized, Browser-Based Docker IDEs for C++ and x86-64 Assembly Education: An Evaluation on the Laptops Students Actually Own

Sean Cuenco
Department of Computer Science, University of Nevada, Las Vegas
Faculty Research Mentor: James Andro-Vasko, Ph.D.
Department of Computer Science, University of Nevada, Las Vegas

Summer Undergraduate Research Fellowship 2026
Office of Undergraduate Research, UNLV

Code, handouts, test script, raw results:
github.com/seancnc003/UNLVDockerIDEs
hub.docker.com/r/seancnc/unlv-cpp-ide
hub.docker.com/r/seancnc/unlv-x86-ide

Introduction

Setting up software can delay the start of a programming course. Several hundred UNLV students a year in introductory C++ (CS 135) and x86-64 assembly (CS 218) must install four programming tools on their own laptops before coursework begins. Some newer Macs cannot run the assembly course's existing virtual machine, a simulated computer with a 19 GB disk. Other universities report the same problem [3, 6].

Purpose/Aim

I packaged the course tools and a code editor into a container that runs on the student's own computer and opens in a browser tab.

Question. On four tested configurations, can the same package run assignments, preserve coursework after deletion, and support debugging?

Measured. Download size and time, editor-opening time, memory use, and build time.

Contribution. Earlier work describes packaged programming workspaces [1, 3, 4]. This project evaluates UNLV's assembly-course package on four configurations and documents its ARM debugger limit.

Methods

Test systems. Three rented virtual computers and my own laptop provided four test configurations (Table 1).

Same test on each. One automated checklist, unchanged on every machine:

1 Download → open the editor → build and run four assignments

2 Test the debugger → delete the container → check the saved coursework

TECHNICAL DETAILS

Machines. AWS m8i.large (Intel) and m8g.large (ARM Graviton); Azure D4s_v5 (Intel, Windows 11 Pro); all rented, under $10 in total. 2021 MacBook Pro M1 Pro, owned. This one command, typed by the student, starts the workspace and connects it to the coursework folder:

```
docker run -p 127.0.0.1:8218:8080 -v ~/UNLV/x86-workspace:/home/coder/workspace
seancnc/unlv-x86-ide
```

Checks. The same download on every machine, confirmed by a file identifier (checksum). All four assignments must build and run without error; the parallel-work assignment (several streams of work at once) must also give its expected answer. The debugger must show the paused program's values and position.

Timing. Editor opening time runs from starting the container until the editor's web page answers the browser. It excludes installing Docker (not measured) and the download (reported separately). Timed on the first start and after replacing the container; second-start results are not shown here.

Memory and builds. Memory is the container's own use (editor and tools; not the browser or Docker), idle and during assignments. Each assignment was built once.

Results

All four tested computers built and ran four assembly-course assignments. Neither ARM computer, including the Apple Silicon Mac, could use the debugger.
ARM is the processor family used in Apple Silicon Macs; most Windows PCs use Intel-style processors.

Assignments ran: 4 of 4 computers

Coursework also survived deleting the workspace on all four.

Debugger worked: 2 of 4 computers

Intel 2 of 2 · ARM 0 of 2
A debugger pauses a program so a student can inspect it. Some assignments need one.

Wait until the editor opens: Intel 0.5–0.6 s, ARM 4.8–6.7 s. **Download:** 552 MB; installing Docker is extra. Times start after Docker is installed and the package downloaded.

Table 1. Results for the four tested computers. MiB is a memory unit close to a megabyte.

Measure	Windows 11 PC Intel, 4 cores, 16 GB rented (Azure)	Linux PC Intel, 2 cores, 8 GB rented (AWS)	Apple Mac M1 Pro (ARM), 16 GB owned MacBook Pro	Linux on ARM Graviton ARM, 2 cores, 8 GB rented (AWS)
Did it work?				
All four assignments built and ran	yes	yes	yes	yes
Parallel-work answer was correct	yes	yes	yes	yes
Coursework kept after workspace deleted	yes	yes	yes	yes
Debugger worked	yes	yes	no	no
How long did it take, and how much memory did it use?				
Download time (seconds)	53	16	45	13
Wait for the editor to open (seconds)	0.6	0.5	4.8	6.7
Highest recorded memory (MiB)	90	not captured*	358	334

*No memory peak was recorded for the Linux PC: its assignments finished before the sampler could take a reading. Timing differences cannot be attributed to processor family alone; the computers differed in provider and resources. Idle memory, build times and the full 13-item checklist are in the repository's results tables.

Figures

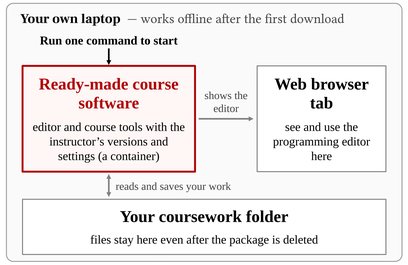
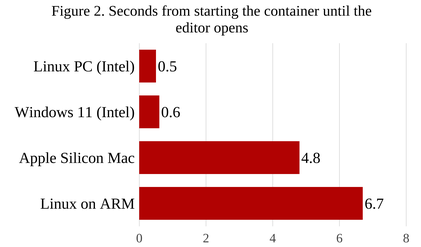


Figure 1. The course software runs on your laptop, appears in your browser, and saves coursework in your own folder. No server is involved.



First-start times after Docker is installed and the package downloaded. ARM machines run the Intel-built package through a translator.

Key Terms

Container a ready-made package of a course's tools. Docker is the free software that runs it.

Virtual machine a whole simulated computer inside a computer; the course's current one is a 19 GB disk.

IDE / editor the application students write programs in; here it opens in a browser tab.

Assembly programming with instructions tied closely to the processor.

Build convert a written program into a form the computer can run.

Debugger a tool that pauses a running program so a student can look inside it.

Working memory temporary space used while software runs, separate from saved files.

Processor families Intel-style and ARM processors understand different basic instructions; Apple Silicon Macs use ARM.

Translator software that runs instructions written for a different processor family. Some features may be unavailable.

Discussion

What it means. On the tested ARM computers, assignments that need the debugger still need an Intel computer or the campus lab. All four tested assignments ran.

Possible explanation. The package uses Intel-style instructions because that is what the course teaches. ARM computers run it through translation software, which may explain the debugger limit. Other differences were not isolated.

Conclusions/Future Research

Conclusion. On four tested computers the package ran all four tested assignments and kept coursework; the two ARM computers could not use the debugger.

Not tested. Student setup success, time saved, or help requests. Four machines are examples, not a survey.

Next: deployment. Offer the packages and setup handouts to students in both courses, starting with instructors who opt in; track setup problems and support requests as they arise; release a revised ARM container that restores the debugger.

Possible applications (not tested here). Courses in statistics, biology, chemistry or engineering that depend on hard-to-install software could distribute similar packages; research tools in those fields are already shared as containers. A separate study could test whether AI coding assistants help instructors build and maintain them.

References/Acknowledgements

[1] D. J. Malan. Containerizing CS50. ITiCSE 2024. [3] S. Valstar, W. G. Griswold, L. Porter. Using DevContainers to Standardize Student Development Environments. ITiCSE 2020. [4] K. Fernalld et al. Lightweight Symphony: Reducing CS Student Anxiety with Standardized Docker Environments. SIGITE 2023. [6] D. P. Harvie et al. Using Virtual Machines to Enhance the Educational Experience. SIGITE 2019. [8] H. Park et al. CodeDive: A Web-Based IDE with Real-Time Code Activity Monitoring. Applied Sciences 15(19), 2025. [12] UNLV Dept. of Computer Science. Student Center: Remote Access and File Storage. tux.cs.unlv.edu, accessed Aug. 2026.

Supported by the UNLV Office of Undergraduate Research Summer Undergraduate Research Fellowship. Every number on this poster comes from the published test script's output files, archived in the project repository.